

Amazon Senior SDE – Kernel Engineer

6-Day Interview Preparation Plan | Aug 8–13, 2026 | Interview: Aug 14, 2026

Goal: Prepare specifically for a Senior SDE – Kernel Engineer screening covering DSA/coding, BSP, Linux Kernel, bootloader, technical depth, debugging, and senior-level cross-team collaboration.

1. Preparation Priorities

Area	Priority	Target Share
Linux Kernel	★★★★★	25%
BSP + Bootloader	★★★★★	20%
C/C++ + DSA	★★★★■	20%
OS / Concurrency / Memory	★★★★■	15%
Debugging / System Design	★★★★■	10%
Leadership / Behavioral	★★★★■	10%

Target: approximately 7–8 focused hours per day. Do not try to learn everything; focus on interview-ready recall, structured explanations, coding speed, and debugging methodology.

2. DAY 1 — Aug 8: C/C++ + Linux Fundamentals + DSA

Morning — C/C++ Deep Dive

- C: pointers and pointer-to-pointer, function pointers, const correctness, volatile, static, extern.
- Structure padding/alignment, bit manipulation, endianness, memory layout, stack vs heap.
- malloc/free, memory corruption, buffer overflow, use-after-free, double free, dangling pointers.
- C++: RAII, Rule of 3/5/0, copy constructor vs assignment, move semantics, lvalue/rvalue.
- Virtual functions, vtable/vptr, virtual destructor, object lifetime, smart pointers.
- unique_ptr vs shared_ptr vs weak_ptr, templates, STL complexity, exception safety.
- std::move, std::forward, const methods, multiple inheritance, diamond problem.

Kernel-specific C

- structs, container_of, offsetof, macros, inline functions, function pointers, callbacks, linked lists, bitmaps, bit manipulation, atomic operations, memory barriers, volatile.
- Why does Linux kernel code use container_of()?
- How would you implement container_of()?
- Why is volatile generally not a synchronization primitive?
- Atomic operation vs memory barrier?

Day 1 Afternoon — DSA

Do 6–8 high-quality problems rather than 30 random problems.

- Arrays/strings: Two Sum, Sliding Window, Longest Substring Without Repeating Characters, Merge Intervals.
- Linked List: Reverse Linked List, Detect Cycle, LRU Cache.
- Trees: Binary Tree BFS/DFS, Lowest Common Ancestor.
- Heap: Top K Elements.
- Graph: BFS/DFS, Number of Islands.

Coding rule: 1) Clarify, 2) explain brute force, 3) optimize, 4) state complexity, 5) code, 6) test edge cases. Senior-level communication while coding matters almost as much as the final code.

Day 1 Evening — Linux OS

- Processes: process, thread, fork(), exec(), wait(), clone(), context switch, scheduler.
- Be able to explain what happens when executing ./myprogram.

```
shell
↓
fork/clone
↓
execve()
↓
ELF loader
↓
dynamic linker
↓
memory mappings
↓
program entry
↓
main()
```

3. DAY 2 — Aug 9: Linux Kernel Deep Dive

Linux Kernel Architecture

```
User Space
|
System Call Interface
|
+-----+
| Kernel |
| Process Management |
| Memory Management |
| VFS |
| Networking |
| Device Drivers |
| IPC |
+-----+
|
Hardware
```

Know: kernel space vs user space, system calls, interrupts, exceptions, kernel threads, modules, and drivers.

System Calls

```
Application
↓
glibc
↓
system call
↓
CPU privilege transition
↓
kernel syscall handler
↓
kernel subsystem
↓
return to user space
```

- What happens internally during read(), open(), fork(), exec(), mmap(), and ioctl()?

Interrupts

```
Device
↓
Interrupt Controller
↓
CPU
↓
Interrupt Handler
```

- IRQ, ISR, hard IRQ, softirq, tasklet, workqueue, threaded IRQ.
- Why can't we perform sleeping operations inside an interrupt handler?

Kernel Synchronization

- spinlock, mutex, semaphore, rwlock, atomic, RCU, memory barriers.
- When use a spinlock instead of a mutex?
- Can you acquire a mutex in interrupt context?
- What happens if two CPUs modify the same kernel data structure simultaneously?

Kernel Memory Management

- virtual memory, physical memory, page, page table, TLB, page fault, buddy allocator, slab/SLUB, kmalloc, vmalloc.
- Know kmalloc() vs vmalloc(), GFP_KERNEL, and GFP_ATOMIC; understand why allocation context matters.

Kernel Debugging

- For a production kernel panic: logs, dmesg, crash dump, stack trace, symbol resolution, git bisect, reproduction, instrumentation, debug symbols, lockdep, KASAN, KMSAN, UBSAN, ftrace, perf, tracepoints.
- Tools: dmesg, journalctl, strace, gdb, perf, ftrace.

4. DAY 3 — Aug 10: BSP + Bootloader + Boot Process

Complete Boot Sequence

```
Power ON
↓
ROM / Boot ROM
↓
Bootloader
↓
Hardware initialization
↓
DRAM initialization
↓
Device Tree / ACPI
↓
Load Kernel
↓
Kernel decompression
↓
Early kernel initialization
↓
MMU
↓
Scheduler
↓
Drivers
↓
Root filesystem
↓
init/systemd
↓
User space
```

Bootloader

- U-Boot architecture, SPL, U-Boot proper, environment variables, boot arguments, kernel image, initrd/initramfs, Device Tree Blob.
- Know concepts/commands: bootcmd, bootargs, load, bootm, booti, bootz.
- Questions: What is U-Boot? Why do we need SPL? Bootloader vs firmware? How does bootloader pass information to Linux?

Device Tree

```
.dts
.dtsi
.dtb

Device Tree Source
↓
dtc
↓
DTB
↓
Bootloader
↓
Linux kernel
↓
Device drivers
```

- compatible, reg, interrupts, clocks, gpio, status, phandle.
- How does Linux know an I2C device exists? Device Tree → compatible property → driver matching → probe().

BSP

Typical BSP components: bootloader, kernel, Device Tree, drivers, board-specific configuration, firmware, build system, root filesystem.

New board bring-up sequence:

1. Verify power/reset
2. Establish UART console
3. Verify boot ROM
4. Verify bootloader
5. Initialize DRAM
6. Load kernel
7. Verify kernel entry
8. Verify DT
9. Early printk/logging
10. Root filesystem
11. Driver bring-up
12. Peripheral validation

5. DAY 4 — Aug 11: OS + Advanced Kernel + Debugging

Memory

```
Virtual Address
↓
Page Table
↓
Physical Address
```

- 4-level page tables, TLB, page fault, demand paging, copy-on-write, mmap, shared memory, kernel virtual memory.
- What happens when a process accesses an unmapped address? What happens during fork()? Why is fork() relatively cheap initially? — Copy-on-write.

Concurrency

- race condition, deadlock, livelock, starvation, priority inversion, atomicity, memory ordering.
- Deadlock example: Thread A lock(A) → lock(B); Thread B lock(B) → lock(A). Fix using global lock ordering.

Scheduling

- scheduler, context switch, run queue, priority, preemption, CPU affinity, SMP, NUMA.
- What happens during a context switch? Why is it expensive? How does Linux schedule threads across CPUs?

Debugging Scenario Practice

- Device randomly reboots after 4 hours.
- Kernel boot hangs after 'Starting kernel...'.
- Driver probe() is never called.
- System has intermittent memory corruption.
- CPU utilization suddenly reaches 100%.
- Deadlock occurs only under high load.
- Kernel panic occurs once every 1000 boots.

For every scenario use:

```

Observe
↓
Collect evidence
↓
Hypothesis
↓
Instrument
↓
Reproduce
↓
Narrow down
↓
Fix
↓
Regression test

```

6. DAY 5 — Aug 12: Full Technical Interview Simulation

Morning — DSA

- 2 medium problems + 1 harder problem. Time yourself; maximum 45 minutes/problem.
- Focus: arrays, hashmap, linked list, tree, graph, heap, binary search, DP basics.

Kernel Mock Interview

- C/C++: virtual functions internally; delete through base pointer; move semantics; stack vs heap; memory alignment; implement memcpy; linked list; memory corruption.
- Linux: fork(), exec(), context switch, system call flow, page fault, interrupt handling, mutex vs spinlock, RCU, kernel memory allocation.
- BSP: boot sequence, U-Boot, Device Tree, driver probing, new-board bring-up, kernel boot failure, driver probe failure.

Amazon Behavioral / Leadership

Prepare 8 strong stories. Amazon Leadership Principles to cover include Customer Obsession, Ownership, Invent and Simplify, Are Right, A Lot, Learn and Be Curious, Insist on the Highest Standards, Think Big, Bias for Action, Earn Trust, Dive Deep, Have Backbone; Disagree and Commit, and Deliver Results.

Use STAR: Situation, Task, Action, Result. Use concrete data/metrics where possible. Do not memorize sentences; remember the structure and facts.

Suggested stories from your experience

- Microsoft OAuth migration — Ownership, Dive Deep, Deliver Results, Customer Obsession, Think Big.
- IBM Confidential Computing / Remote Attestation — Technical depth, Ownership, Dive Deep, Invent & Simplify.
- Kata Containers / RHEL / s390x debugging — Dive Deep, Highest Standards, problem solving, Ownership.
- Major production/live-site issue — Bias for Action, Ownership, Deliver Results.
- Disagreement with architect/team — Have Backbone; Disagree and Commit.
- Cross-team project — Earn Trust, Customer Obsession, Deliver Results.
- Failure story — what happened, what went wrong, why, learning, what changed, result.
- Simplification/automation story — simplifying a complex process.

7. DAY 6 — Aug 13: Final Revision — No New Topics

Morning — C/C++ Rapid Revision

- volatile, const, static, extern, pointer, double pointer, function pointer, memory alignment, padding, RAII, move semantics, virtual function, vtable, smart pointers, atomic, memory ordering.

11 AM–1 PM — Kernel Rapid Revision

- Process, thread, scheduling, interrupt, system call, virtual memory, page fault, synchronization, kernel memory, driver, VFS.

2 PM–4 PM — BSP/Bootloader Rapid Revision

```
Power ON
↓
ROM
↓
SPL
↓
U-Boot
↓
DRAM
↓
DTB
↓
Kernel
↓
initramfs
↓
drivers
↓
rootfs
↓
systemd
```

4 PM–6 PM — Final DSA: only 3 problems. Do not exhaust yourself.

Evening — Behavioral: review your 8 stories. For each remember Situation, Task, 3–5 Actions, Metrics, Result, Learning. Do not memorize sentences.

8. Highest-Priority Interview Questions

C/C++

- volatile vs atomic

- stack vs heap
- memory alignment
- RAII
- move semantics
- virtual function/vtable
- smart pointers
- race conditions
- memory corruption
- implement memcpy

Linux

- fork()
- exec()
- system call
- context switch
- page fault
- interrupt vs exception
- hard IRQ vs softirq
- mutex vs spinlock
- RCU
- kernel memory allocation
- kmalloc vs vmalloc
- GFP_KERNEL vs GFP_ATOMIC
- kernel/user-space boundary
- scheduler
- process vs thread

BSP / Boot

- complete boot process
- bootloader responsibilities
- U-Boot vs SPL
- Device Tree
- DTS vs DTB
- driver matching
- probe()
- new-board bring-up
- kernel boot debugging
- driver probe debugging

9. Senior-Level Positioning Strategy

Your background is not that of a fresh kernel candidate. You have substantial systems experience across Samsung (C++/system development), IBM (Linux, C++, Confidential Computing, Kubernetes, s390x), and Microsoft (Windows/C++, C#/cloud, large-scale authentication migration).

Position yourself as a systems engineer who has worked across OS, low-level C/C++, Linux, cloud infrastructure and large-scale production systems, and who can go from hardware/OS-level debugging to distributed-system-level problems.

Your Kata/s390x debugging experience is especially valuable because it gives you real examples of kernel/runtime/boot/debugging problems rather than purely theoretical knowledge.

10. Exact Daily Schedule

Time	Activity
09:00–11:30	Core technical topic
11:30–12:00	Break
12:00–14:00	DSA / C++
14:00–15:00	Lunch
15:00–17:30	Kernel/BSP/OS
17:30–18:00	Break
18:00–19:30	Interview questions / debugging
20:30–21:30	Amazon behavioral

11. Six-Day Calendar

Date	Main Goal
Aug 8	C/C++ + DSA + Linux fundamentals
Aug 9	Linux Kernel deep dive
Aug 10	BSP + Bootloader + Device Tree
Aug 11	Memory + concurrency + scheduling + debugging
Aug 12	Full technical mock + behavioral
Aug 13	Rapid revision + DSA + Amazon stories
Aug 14	INTERVIEW

12. Final Coaching Strategy

Use the plan interactively: practice one question at a time as if in the interview; after each answer, assess what was good, what was missing, and what a Senior SDE-level answer should sound like. Prioritize DSA, kernel, BSP, boot, and behavioral stories rather than passively reading notes.

Interview Mindset

- Think aloud during technical questions.
- For debugging, start with evidence rather than guessing.
- For design/architecture, state assumptions and trade-offs.
- For coding, clarify requirements and test edge cases.
- For behavioral answers, emphasize your individual contribution, decisions, measurable outcomes, and lessons learned.
- At Senior level, demonstrate ownership, technical depth, cross-team communication, and the ability to drive ambiguous problems to closure.